

## Conteúdo

- **Introdução**
  - Motivação, Objetivo, Definição, Características Básicas e Histórico
- **Conceitos Básicos**
  - Neurônio Artificial, Modos de Interconexão
- **Processamento Neural**
  - Recall e Learning
- **Regras de Aprendizado**
  - Regra de Hebb, Perceptron, Back Propagation, Hopfield, Competitive Learning, RBF, etc.

## Regra de Hebb

- **Declaração de Hebb:**

“Quando um axônio de uma **célula A** está próximo o suficiente de **excitar** a **célula B** e repetidamente ou persistentemente participa da ativação desta, um processo de crescimento ou mudança metabólica ocorre em uma ou ambas as células, de tal forma que a **eficiência** de **A** em **ativar B** é **augmentada**”.

Portanto, a cada apresentação do padrão a saída fica mais reforçada.

## Algoritmos de Aprendizado

- Regra de Hebb
- Perceptron
- Delta Rule (Least Mean Square)
- Back Propagation
- Hopfield
- Competitive Learning
- Radial Basis Functions (RBFs)

## Regra de Hebb

- **Em termos práticos:**
  - Se dois neurónios em cada lado de uma sinapse (conexão) **são ativados simultaneamente** (sincronamente), então a “força” daquela **sinapse deve ser aumentada**.
  - Se dois neurónios em cada lado de uma sinapse **são ativados assincronamente**, então aquela **sinapse deve ser enfraquecida**.

## Regra de Hebb

- **Declaração de Hebb:**

“Quando um axônio de uma **célula A** está próximo o suficiente de **excitar** a **célula B** e repetidamente ou persistentemente participa da ativação desta, um processo de crescimento ou mudança metabólica ocorre em uma ou ambas as células, de tal forma que a **eficiência** de **A** em **ativar B** é **augmentada**”.

## Regra de Hebb

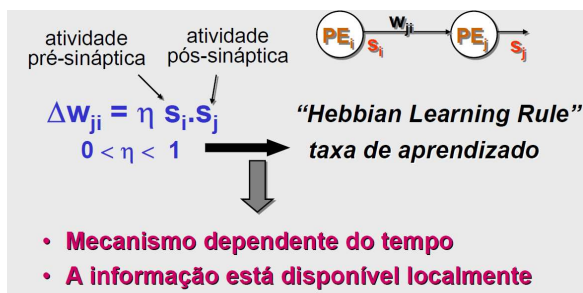
- **Conclusão:**

A **modificação** nas sinapses tem relação com a **correlação entre as atividades pré-sinápticas e pós-sinápticas**.

Correlação + → o valor do peso **aumenta**  
Correlação - → o valor do peso **diminui**

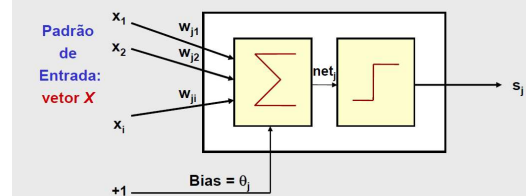
A saída é reforçada a cada apresentação do padrão → **padrões frequentes terão maior influência no vetor de pesos do neurônio.**

## Regra de Hebb



## Perceptron

Na sua forma mais simples o modelo do processador consiste de:



## Algoritmos de Aprendizado

Os algoritmos podem ser definidos através das seguintes características:

- ❖ Regra de Propagação
- ❖ Função de Ativação
- ❖ Topologia
- ❖ Regra de Aprendizado -  $\Delta w$

## Perceptron

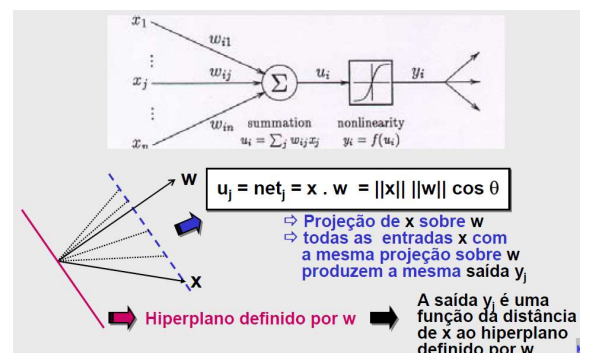
### • Características Básicas:

- Regra de Propagação  $\rightarrow net_j = \sum x_i \cdot w_{ji} + \theta_j$
- Função de Ativação  $\rightarrow$  **Degrau**
- Topologia  $\rightarrow$  **Uma única camada de processadores.**
- Algoritmo de Aprendizado  $\rightarrow$  **Supervisionado:** (geralmente)  $\Delta w_{ji} = \eta \cdot x_i \cdot (t_j - s_j)$
- Valores de Entrada/Saída  $\rightarrow$  **Binários {-1,1}**

## Algoritmos de Aprendizado

- Regra de Hebb
- **Perceptron**
- Delta Rule (Least Mean Square)
- Back Propagation
- Radial Basis Functions (RBFs)
- Competitive Learning
- Hopfield

## Análise Geométrica



## Análise Geométrica

Logo, superfícies de **saída constante** são **hiperplanos perpendiculares a  $w$**

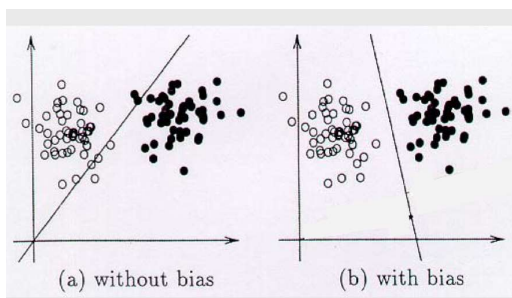
$\sum_i w_i x_i = 0 \Rightarrow$  define um **hiperplano** passando **pela origem**

Incluindo o bias  $\Rightarrow \sum_i w_i x_i + \theta_j$   
desloca-se o hiperplano da origem

## Algoritmo de Aprendizado

- ❶ Iniciar os pesos sinápticos com valores randômicos e pequenos;
- ❷ Aplicar um padrão de entrada, com seu respectivo valor desejado de saída ( $t_j$ ), e verificar a saída da rede ( $s_j$ );
- ❸ Calcular o erro na saída:  $e_j = t_j - s_j$  (geralmente)
- ❹ Se  $e_j = 0$ , voltar ao passo 2;  
Se  $e_j \neq 0$ , atualizar os pesos:  $\Delta w_{ji} = \eta \cdot x_i \cdot e_j$
- ❺ Voltar ao passo 2.

## Análise Geométrica



## Algoritmo de Aprendizado

- Outras regras simples de aprendizado: (como  $t, y \in \{-1, +1\}$ , todas são equivalentes)

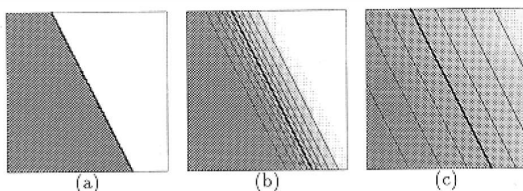
$$\checkmark \Delta w_{ji} = \begin{cases} 2 \cdot \eta \cdot t_j \cdot x_i & \text{se } t_j \neq y_j \\ 0 & \text{caso contrário} \end{cases}$$

$$\checkmark \Delta w_{ji} = \eta \cdot (1 - t_j \cdot y_j) \cdot t_j \cdot x_i$$

$$\checkmark \Delta w_{ji} = \eta \cdot (t_j - y_j) \cdot x_i$$

## Análise Geométrica

**Função de Ativação**  $\Rightarrow$  controla como a **saída  $y$**  varia com a **distância de  $x$  ao hiperplano definido por  $w$**



**Função Degrau**  $\Rightarrow$  divide o espaço de entrada com um hiperplano

**Função Sigmoid**  $\Rightarrow$  transição suave  
 $||\partial y / \partial x|| = f'(\text{net}) ||w||$

## Algoritmo de Aprendizado

### ■ IMPORTANTE:

- **não ocorre variação** no peso se a saída estiver **correta**;
- caso contrário, cada **peso é incrementado** de  $2\eta$  quando a **saída é menor** que o target e **decrementado** de  $2\eta$  quando a **saída é maior** que o target.

## O Problema do OU-Exclusivo

Rosenblatt (1962) provou que:

*Uma rede Perceptron é capaz de Aprender tudo que puder Representar.*

**Representação** → refere-se à habilidade do sistema neural de representar (*simular*) uma função específica.

**Aprendizado** → refere-se à existência de um procedimento sistemático de aquisição de conhecimento (*ajuste dos pesos*), de forma a produzir a função desejada

## O Problema do OU-Exclusivo

**Conclusão:**

– mudando-se os valores de  $w_1$ ,  $w_2$  e  $\theta$ , muda-se a *inclinação* e a *posição* da reta.



– Entretanto, é *impossível* achar uma reta que divida o plano de forma a separar os pontos  $A_1$  e  $A_2$  de um lado e  $A_0$  e  $A_3$  de outro.



– Redes de **1 única camada** só representam *funções linearmente separáveis!*

## O Problema do OU-Exclusivo

- Minsky & Papert provaram (*Perceptrons 1969*) que existem sérias restrições sobre o que as redes Perceptron são capazes de *Representar*.



- Por exemplo, as redes Perceptron **NÃO** são capazes de *Representar* a função **OU-Exclusivo**

## O Problema do OU-Exclusivo

**Conclusão:**

– mudando-se os valores de  $w_1$ ,  $w_2$  e  $\theta$ , muda-se a *inclinação* e a *posição* da reta.

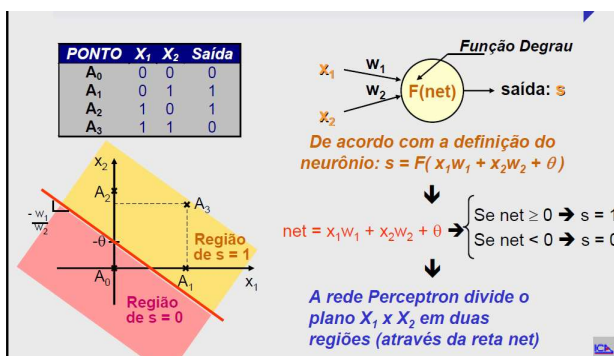


– Entretanto, é *impossível* achar uma reta que divida o plano de forma a separar os pontos  $A_1$  e  $A_2$  de um lado e  $A_0$  e  $A_3$  de outro.

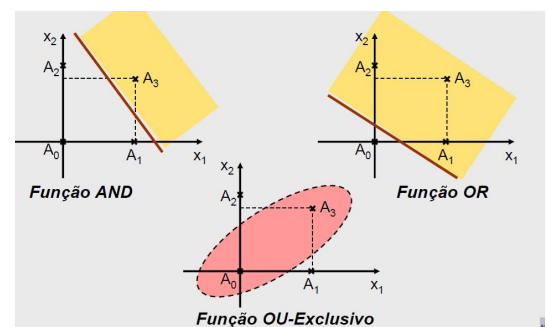


– Redes de **1 única camada** só representam *funções linearmente separáveis!*

## O Problema do OU-Exclusivo



## O Problema do OU-Exclusivo



## O Problema do OU-Exclusivo

Minsky & Papert provaram que este problema pode ser solucionado adicionando-se uma outra **camada intermediária** de processadores.



**Multi-Layer Perceptrons**

## O Problema do OU-Exclusivo

### • Em termos vetoriais:

$$\text{Camada Escondida} \Rightarrow \text{NET}_1 = S_1 \cdot W_1 \\ S_1 = k_1 \text{NET}_1$$

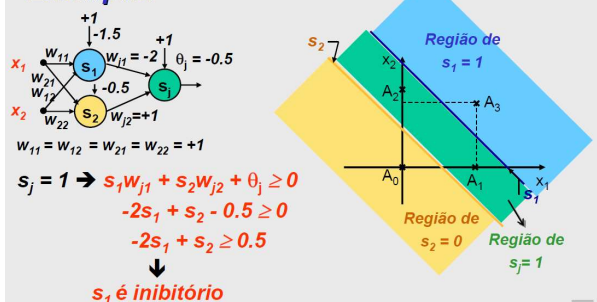
$$\text{Camada de Saída} \Rightarrow S_2 = k_2 \text{NET}_2 = k_2(S_1 \cdot W_2) \Rightarrow \\ S_2 = k_2 [(k_1 \text{NET}_1) W_2] \Rightarrow \\ S_2 = k_2 [(k_1 S_1 W_1) W_2] \Rightarrow \\ S_2 = k_2 k_1 (S_1 W_1) W_2 \Rightarrow \\ S_2 = K \cdot S_1 \cdot (W_1 \cdot W_2) \\ S_2 = K \cdot S_1 \cdot W$$



**Equivalente a uma única camada**

## O Problema do OU-Exclusivo

### Exemplo:



Fim do Perceptron

## O Problema do OU-Exclusivo

### • Observação:

Redes Neurais de **múltiplas camadas** só oferecem **vantagens** sobre as de uma **única camada** se existir uma **função de ativação não-linear** entre as camadas!